LLMs as Acquisition Policies for Finite-Pool Materials Optimization: A Controlled Study

Dino–Rober Demir
dino-rober.demir@polytechnique.org

Florian Le Bronnec
florian.lebronnec@riken.jp

Rio Yokota
rio.yokota@riken.jp

RIKEN Center for Computational Science, Tokyo, Japan

Abstract

Discovering materials with desirable properties often requires searching large candidate spaces while experimental or computational evaluations remain costly. Active learning addresses this challenge by using previous observations to select which candidate to evaluate next, typically through probabilistic surrogate models. We investigate whether open-weight large language models (LLMs) can serve as standalone acquisition policies in this setting. We evaluate five LLMs across four retrospective finite-pool materials optimization tasks under different candidate-presentation strategies and compare them with random selection and conventional Gaussian-process methods. LLM policies generally reach the global optimum in fewer iterations than random selection, indicating that they provide a useful acquisition signal without task-specific training. Their performance relative to Gaussian-process methods is mixed: conventional acquisition performs better on most tasks, while LLMs match or outperform it in some settings. Performance varies substantially across tasks, models, initializations, and candidate presentations, with no LLM approach performing best across all tasks. Overall, open-weight LLMs show potential as acquisition policies for finite-pool materials search, although their reliability remains sensitive to the task and to how candidates and scientific context are presented.

1 Introduction

Materials science has long been a thriving field of research, and the discovery of new materials is a key driver of technological progress [de Pablo et al., 2019, Jain et al., 2013]. However, conventional experimental and computational screening remains costly and time-consuming [Schmidt et al., 2019, Pyzer-Knapp et al., 2022]. In the last decade, machine learning has emerged as a promising way to address these challenges by predicting materials properties and guiding the search for new materials [Butler et al., 2018, Ramprasad et al., 2017]. In this work, we evaluate whether pretrained LLMs can translate their learned priors into useful acquisition policies for AI-guided materials discovery and study their reliability relative to established materials active-learning methods.

Active learning and Bayesian optimization are standard tools for materials discovery when experimental or computational labels are expensive [Lookman et al., 2019]. In a typical surrogate-based closed-loop experiment, an optimizer observes a small set of labeled candidates, fits a predictive model, and selects the next candidates to evaluate by maximizing an acquisition function that captures the trade-off between exploration and exploitation over the candidate pool. Gaussian processes

Preprint.

(GPs), together with acquisition functions such as expected improvement (EI) or upper confidence bound (UCB), remain strong baselines in Bayesian optimization because they explicitly model both predicted performance and epistemic uncertainty [Kushner, 1964, Srinivas et al., 2010, Murphy, 2022].

Large language models (LLMs) offer a different route to candidate selection. Recent reasoning-focused LLMs have demonstrated strong performance on scientific reasoning benchmarks [OpenAI, 2024, Guo et al., 2025]. The extent of their knowledge, their reasoning capabilities, and their applicability to materials science are still being explored. These capabilities make LLMs attractive for sequential candidate selection, but their reliability across materials problems remains unclear.

We conduct this evaluation on controlled retrospective materials active-learning benchmarks, where LLMs are prompted with the observed history, a candidate list, and a natural-language description of the scientific objective. At each iteration, the LLM policy selects exactly one unobserved composition from a finite candidate pool, receives the corresponding property value from the dataset, and repeats until it selects the globally optimal candidate. This protocol allows direct comparison between classical GP acquisition functions, standalone LLM selection, and LLM acquisition variants under identical experimental conditions.

We organize the study around three questions:

- How competitive and reliable are LLM acquisition policies compared with GP-EI?
- How do candidate presentation and materials-specific semantic context affect LLM selection?
- How do LLM-selected candidates compare to those selected by GP-based Bayesian optimization?

We address these questions by evaluating five open-weight LLMs across four materials optimization tasks and a range of experimental conditions. Across these experiments, we observe a useful but variable acquisition signal and examine how its reliability is shaped by the task, candidate presentation, and semantic context.

2 Related Work

Active learning for materials discovery. Materials discovery remains costly, motivating machine-learning approaches to predict properties and guide candidate selection [de Pablo et al., 2019, Schmidt et al., 2019, Butler et al., 2018]. Retrospective materials benchmarks provide a controlled setting for evaluating active-learning methods by treating a labeled dataset as an oracle [Wang et al., 2022]. While they do not fully reproduce experimental deployment, they allow repeated trials from different initializations and direct comparison between acquisition policies. This makes them particularly well suited to our goal: isolating how LLM-based candidate selection behaves relative to established acquisition strategies, without run-to-run experimental variation or changes to the candidate pool.

LLMs for active learning and optimization. LLMs have been incorporated into active-learning and optimization pipelines in several roles. In machine learning, LLMs have been used to select examples for training downstream models [Bayer et al., 2026, Parkar et al., 2024] and to prune unlabeled pools before standard active-learning acquisition [Azeemi et al., 2026]; Xia et al. [2025] review this broader literature. In black-box Bayesian optimization, Chang et al. [2025] introduce LLINBO, a hybrid framework that combines LLM-suggested query points with a statistical surrogate. These studies illustrate how LLMs can support data acquisition, either by identifying useful training examples or by complementing conventional optimizers. However, they primarily address language tasks or general black-box optimization rather than materials discovery specifically.

LLMs for materials optimization. Prior work in chemistry and materials optimization has coupled LLMs with probabilistic surrogates [Kristiadi et al., 2024, Ranković and Schwaller, 2025] and used LLM-generated hypotheses or tool-enabled agents to explore large implicit design spaces, with or without Bayesian optimization [Cissé et al., 2025, 2026]. Unlike these methods, we focus on standalone LLM acquisition from explicit finite candidate pools. The most closely related study by Wang et al. [2026] uses a generation-and-matching approach (see Section C.1) within small candidate pools containing 73 to 323 points with closed-weight LLMs. We evaluate this protocol on open-weight LLMs and compare it with other LLM policies.

3 Background

Pool-based active-learning protocol. Let $\mathcal{X} = \{x_i\}_{i=1}^N$ be a finite pool of candidate materials with known labels $\{y_i\}_{i=1}^N$, hidden from the policy until queried. For each dataset and acquisition-policy configuration, candidate $x_i \in \mathbb{R}^d$ denotes the representation provided to the policy, where d depends on the dataset and input representation, and $y_i \in \mathbb{R}$ is its scalar label. The goal is to maximize the target value y among $\{y_i\}_{i=1}^N$. At iteration t, the policy observes

$$\mathcal{D}_t = \{(x_j, y_j) : j \in \mathcal{I}_t\}, \tag{1}$$

where $\mathcal{I}_t$ is the set of previously queried indices, and selects one unobserved candidate

$$i_t \in \{1, \ldots, N\} \setminus \mathcal{I}_t. \tag{2}$$

The oracle returns y_{i_t}, and the observed set is updated to $\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(x_{i_t}, y_{i_t})\}$ (and $\mathcal{I}_{t+1} = \mathcal{I}_t \cup \{i_t\}$). On a fixed seed, every method is initialized with the same single initial point. Moreover, each subsequent acquisition adds exactly one newly queried index to $\mathcal{I}_t$ and the corresponding candidate-label pair to $\mathcal{D}_t$.

Gaussian-process baseline. We compare against GP-based Bayesian optimization baselines. At each iteration, the GP is fit on the observed data $\mathcal{D}_t$, producing a posterior mean $\mu_t(x)$ and posterior standard deviation $\sigma_t(x)$ for each unobserved candidate. We use the Expected Improvement (EI) acquisition function [Jones et al., 1998]. It selects:

$$i_t^{\mathrm{EI}} \in \operatorname*{arg\,max}_{i \in \{1,\ldots,N\} \setminus \mathcal{I}_t} \mathrm{EI}_t(x_i), \tag{3}$$

where

$$\mathrm{EI}_t(x) = (\mu_t(x) - b_t)\Phi(z_t(x)) + \sigma_t(x)\phi(z_t(x)), \quad z_t(x) = \frac{\mu_t(x) - b_t}{\sigma_t(x)}, \quad b_t = \max_{j \in \mathcal{I}_t} y_j, \tag{4}$$

When $\sigma_t(x) = 0$, we set $\mathrm{EI}_t(x) = (\mu_t(x) - b_t)_+$. Here Φ and ϕ denote the standard normal CDF and PDF. EI combines predicted improvement over the incumbent with posterior uncertainty. In the canonical form used here, it does not require an explicit exploration coefficient such as the β parameter in GP-UCB.

Random baseline. We also compare our acquisition policies to a random baseline, in which the points are sampled randomly among the remaining unlabeled data:

$$i_t^{\mathrm{Random}} \sim \mathcal{U}(\{1, \ldots, N\} \setminus \mathcal{I}_t). \tag{5}$$

4 LLM-Based Candidate Selection

LLM acquisition policies. An LLM acquisition policy π_{LLM} receives the observed history, a candidate list, and task instructions, then returns the index of one candidate to evaluate next. Unlike GP-EI, which assigns an explicit acquisition score to every remaining candidate after fitting the surrogate, the LLM policies considered here make direct, potentially list-dependent choices and do not construct a pool-wide acquisition-score vector. Obtaining a separate LLM assessment for every candidate would require additional inference proportional to the pool size and is not necessary here. It selects:

$$i_t^{\mathrm{LLM}} \sim \pi_{\mathrm{LLM}}\left(\cdot \mid \mathcal{D}_t, \mathcal{C}_t, p\right), \tag{6}$$

where $\mathcal{C}_t$ is an ordered presentation to the LLM of unlabeled data, i_t^{LLM} denotes its final selected candidate index rather than the output of an individual LLM call, and p is the prompt and decoding configuration. Unless otherwise specified, we use materials-aware prompts: candidates are represented using semantically meaningful feature labels, such as element names and target-property names, together with a materials-specific description of the task. These system prompts depend on the dataset and optimization objective; examples are provided in Appendix B.4.

Table 1: Retrospective optimization tasks.

Task	Candidates	Features	Target
`Kerr Rotation`	921	3	Kerr rotation (mrad)
`Coercivity`	921	3	coercivity (mT)
`Matbench Steels`	312	14	yield strength (MPa)
`Electrostrain`	81	15	electrostrain (%)

Candidate-selection protocols. Each candidate-selection protocol induces an LLM-based acquisition policy over the remaining pool. Our primary protocol is whole-pool selection: it presents all remaining candidates to the model in a single, randomly ordered list and asks the model to return the displayed index of the candidate it considers most promising. A protocol may instead involve multiple nested LLM calls, but it always yields a single candidate index for evaluation. We introduce the batch-tournament protocol alongside its analysis in Section 7; implementation details for both protocols are provided in Appendix B.

5 Experimental setup

Datasets. We evaluate our methods on four optimization tasks spanning three materials datasets. Each task is defined by a finite candidate pool in which every candidate has a vector-valued representation and a scalar objective to maximize.

`Fe-Co-Ni`. The `Fe-Co-Ni` combinatorial thin-film benchmark introduced by Wang et al. [2022] contains 921 experimentally measured ternary compositions, i.e., $\mathrm{Fe}_x\mathrm{Co}_y\mathrm{Ni}_{1-x-y}$. Although the original library also includes X-ray diffraction data and measured magnetic properties, we use composition-only inputs. We consider two target properties: Kerr rotation in mrad and magnetic coercivity in mT. They define two separate optimization tasks called `Kerr Rotation` and `Coercivity`, respectively. The original benchmark describes coercivity as the more challenging target because of its more complex objective landscape.

`Matbench Steels`. Introduced as `matbench_steels` in the Matbench v0.1 benchmark by Dunn et al. [2020], this dataset is defined as a supervised regression task, with yield strength in MPa as the target property. It contains 312 composition-only entries with 14 features. We follow the retrospective pool-based optimization framing also used for `Matbench Steels` by Wang et al. [2026], in which we treat the labeled dataset as a finite oracle and ask each acquisition policy to maximize yield strength.

`Electrostrain`. This dataset is derived from the $BaTiO_3$-based piezoelectric active-learning study of Yuan et al. [2018], also reviewed by Lookman et al. [2019]. It combines the 61 initial measurements with the 20 compositions synthesized over five active-learning iterations, yielding a finite pool of 81 candidates. The original prospective search considered solid solutions of the form $(\mathrm{Ba}_{1-x-y}\mathrm{Ca}_x\mathrm{Sr}_y)(\mathrm{Ti}_{1-u-v}\mathrm{Zr}_u\mathrm{Sn}_v)\mathrm{O}_3$; the initial measurements additionally contain a small number of Cd-containing compositions. Each candidate is represented by eight cation-fraction columns (Ba, Ca, Sr, Cd, Ti, Zr, Sn, and Hf) and seven auxiliary physicochemical descriptors, for 15 input features in total; Hf is identically zero in the observed pool. The objective is to maximize the bipolar electrostrain measured at $20\,\mathrm{kV\,cm^{-1}}$. Further details are provided in Appendix B.2.

Models. To facilitate reproducibility and limit inference costs, we restrict our experiments to five open-weight models spanning three model families and several scales. This diversity allows us to assess the sensitivity of our findings to model family and scale. These include Gemma 4 31B, DeepSeek-V4-Flash (284B-A13B), Qwen3.5-27B, Qwen3.5-35B-A3B, and Qwen3.5-397B-A17B [1] [Gemma Team, 2026, DeepSeek-AI et al., 2026, Qwen Team, 2026]. Further model and inference details are provided in Appendix B.1.

[1] Throughout the whole paper, * and † near Qwen3.5-397B-A17B runs results denote proxy runs due to inference costs; see Appendix B.1

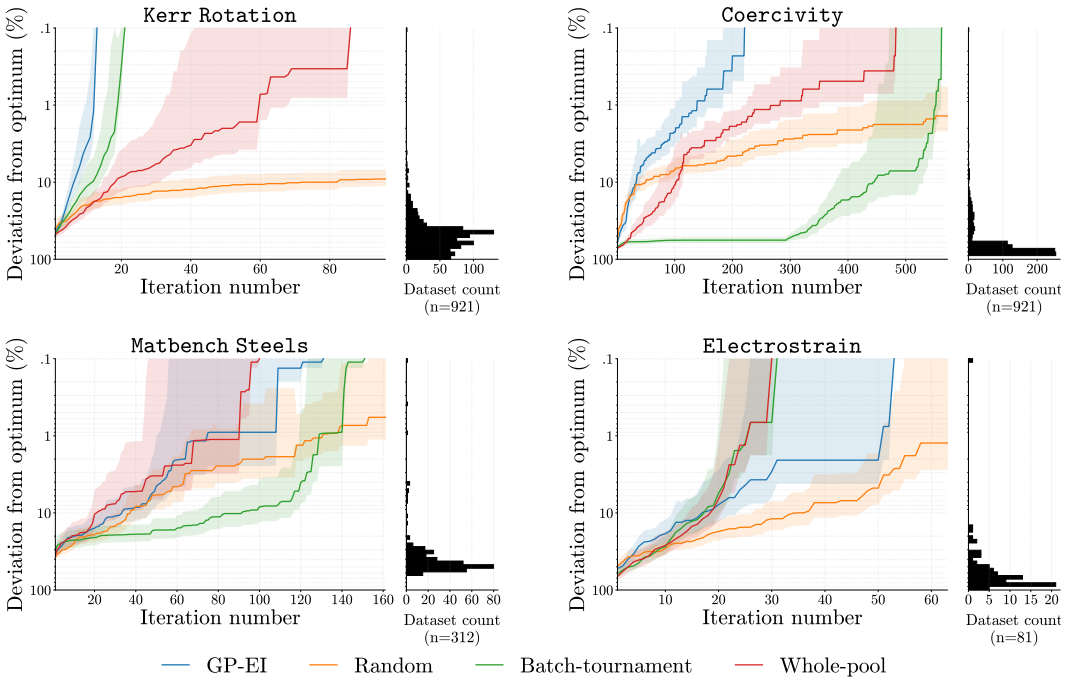


Figure 1: Best-so-far optimization trajectories using Gemma 4 31B for the LLM policies. At iteration t, each curve shows the mean percentage gap $100(y^\star - b_t)/|y^\star|$ across the 25 shared initialization seeds. Shading denotes pointwise normal-approximation 95% confidence intervals, computed as $\bar{g}_t \pm 1.96 s_t/\sqrt{25}$. We show Gemma 4 31B because it ranks among the strongest LLMs across all four tasks.

6 Whole-Pool LLM Acquisition

We first evaluate whole-pool LLM selection against random selection and GP-EI. Table 2 additionally reports batch-tournament results for completeness; these are analyzed in the next section.

Performance and variability. Table 2 compares whole-pool LLM selection with GP-EI and random selection. GP-EI achieves the lowest mean iterations to optimum on three of the four tasks: `Kerr Rotation`, `Coercivity`, and `Matbench Steels`. `Electrostrain` is the exception, with three of the five LLMs requiring fewer iterations on average than GP-EI. Every whole-pool LLM also achieves a lower mean than random selection on every task. Whole-pool selection therefore provides a useful acquisition signal, although it does not consistently match GP-EI across tasks. Variability also tends to follow similar trends to those of mean iterations to optimum, although the magnitude of the deviation depends strongly on the model and task.

Consistency. Whole-pool model rankings are not consistent across tasks. DeepSeek-V4-Flash achieves the lowest mean iterations to optimum on `Electrostrain`, but the highest on `Kerr Rotation` and `Matbench Steels`. By contrast, Gemma 4 31B remains near the top across all four tasks. Neither model scale nor architecture provides a clear ordering even within the Qwen3.5 family: Qwen3.5-27B consistently outperforms Qwen3.5-35B-A3B, while the available results for Qwen3.5-397B-A17B show no systematic advantage over Qwen3.5-27B. Therefore, neither single-task performance nor nominal model scale yields a consistent cross-task ranking. These reversals motivate examining whether candidate presentation, rather than model choice alone, changes acquisition performance.

Table 2: Iterations to the global optimum over 25 initializations (mean ± standard deviation), except for random selection which is initialized over 1000 seeds. Lower is better. Baselines are shown once because they do not depend on an LLM presentation protocol.

Model	`Kerr Rotation`	`Coercivity`	`Matbench Steels`	`Electrostrain`
RANDOM	462.48 ± 262.24	448.09 ± 269.63	159.31 ± 91.28	39.89 ± 23.00
GP-EI	8.40 ± 2.78	91.80 ± 62.77	42.72 ± 27.24	18.48 ± 14.53
Whole-pool selection				
GEMMA4	31.28 ± 17.24	209.88 ± 134.05	44.16 ± 23.36	14.36 ± 7.71
QW27B	27.08 ± 13.59	195.20 ± 114.39	48.24 ± 23.43	19.44 ± 10.40
QW35B	45.72 ± 20.28	289.88 ± 127.44	50.36 ± 24.89	27.64 ± 18.15
QW397B	23.48 ± 10.40*	258.24 ± 151.68*	53.32 ± 32.90*	16.84 ± 5.93
DS	110.64 ± 85.31	236.28 ± 210.72	87.60 ± 52.10	13.88 ± 9.98
Batch-tournament ($b = 20$)				
GEMMA4	13.12 ± 5.12	389.64 ± 92.00	78.08 ± 49.83	14.80 ± 6.50
QW27B	14.28 ± 7.33	468.64 ± 50.68	64.52 ± 32.94	18.04 ± 5.33
QW35B	22.00 ± 8.73	379.80 ± 110.27	52.68 ± 20.12	20.36 ± 9.89
QW397B	14.48 ± 7.54	178.80 ± 82.87†	81.96 ± 41.10	15.08 ± 6.92
DS	16.88 ± 5.52	241.84 ± 52.10	32.88 ± 31.71	12.36 ± 4.71

7 Batch-Tournament LLM Acquisition

Batch-tournament LLM selection. LLMs are known to struggle with long contexts [Liu et al., 2024]. To control the number of candidates shown in each call, batch-tournament selection randomly splits the unobserved pool into batches of size b. The LLM selects one winner from each batch, after which the winners are rebatched and the process repeats until the remaining set is small enough for a final LLM selection (see Figure 6). This protocol gives every candidate an opportunity to enter the tournament while limiting the size of each individual comparison. When the batch size equals the pool size, it reduces to whole-pool selection. Unless otherwise specified, we use (b=20) across all experiments, so the LLM receives at most 20 candidates per call.

Small batch-tournament selection behaves differently from whole-pool selection. Table 2 shows that, on `Kerr Rotation` and `Electrostrain`, batch-tournament selection achieves a lower mean iterations to optimum and a lower standard deviation across almost every model. However, the same cannot be said on `Matbench Steels` and `Coercivity`, which are the two tasks that generally take more iterations to converge. Overall, comparable mean iterations to optimum tends to yield higher variability for the whole-pool selection compared to a small batch-sized tournament, which is closer to that of GP-EI. Regardless, neither selection policy uniformly dominates the other across the

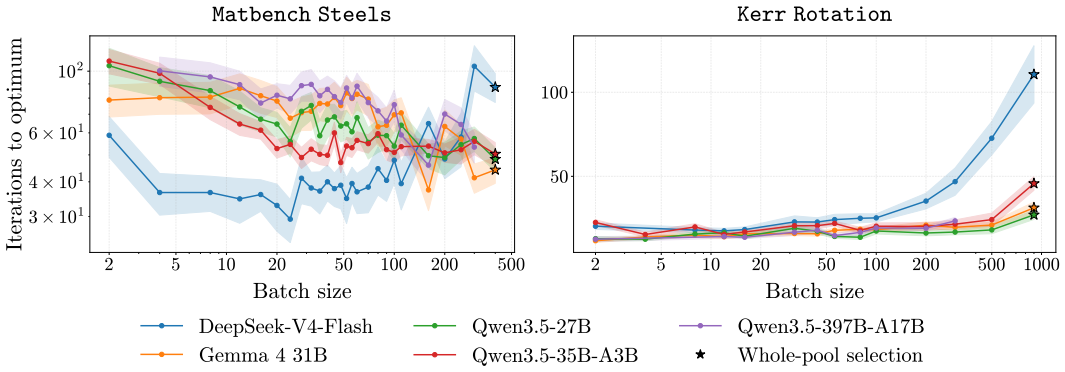
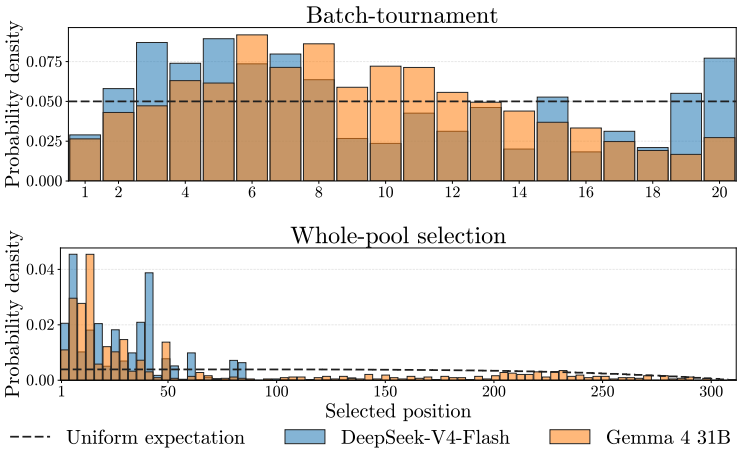


Figure 2: Batch-size sensitivity of tournament selection on `Matbench Steels` and `Kerr Rotation` for runs of 25 seeds. Curves report the mean number of iterations to the optimum across seeds, with uncertainty shown shaded as the mean $\pm$ std$/\sqrt{n_{\text{seeds}}}$.



(a) Position distributions under batch-tournament and whole-pool selection.

Models	$D_{\mathrm{KL}}(q \,\|\, u)$
GEMMA4	1.02
DS	1.47
QW27B	0.852
QW35B	0.747
QW397B	1.36*

(b) Whole-pool position bias on `Matbench Steels`.

Figure 3: Position bias for candidates selected by DeepSeek-V4-Flash and Gemma 4 31B on `Matbench Steels`. Dashed lines in panel (a) indicate the uniform-selection expectation, which may not be horizontal because the pool shrinks across iterations. Panel (b) reports $D_{\mathrm{KL}}(q \,\|\, u)$ for all evaluated models.

four tasks. The trajectories in Figure 1 show that even in early stages, sometimes batch-tournament is one of the best policies and sometimes it is even worse than random selection for Gemma 4 31B.

Batch-size effects are task-dependent, with model-specific deviations. Figure 2 shows that no batch size performs consistently well across tasks. For the Qwen3.5 models and Gemma 4 31B, increasing the batch size generally reduces the mean iterations to optimum on `Matbench Steels`, but increases it on `Kerr Rotation`. Smaller batches therefore tend to work better on the latter task, whereas the first one benefits from a wider view of the candidate pool (`Coercivity` follows a trend similar to that of `Matbench Steels`, and `Electrostrain` similar to that of `Kerr Rotation`, see Figure 7). The pattern nevertheless also depends on the model: DeepSeek-V4-Flash follows the broad trends at small and intermediate batch sizes, but deteriorates sharply at the largest sizes. This deviation may be related to its stronger position bias, examined in Section 8.1.

Additional candidate-selection protocols performance. Similar experiments were conducted with a generation-and-matching protocol. The results are reported in Appendix C.1.

8 Diagnostics of LLM Acquisition Behavior

8.1 Position bias

LLM performance on long-context information retrieval and question-answering tasks has been shown to depend strongly on the position of relevant information, often degrading when that information occurs in the middle of the context [Liu et al., 2024]. We therefore examine where in the batch each selected candidate occurs and quantify position bias as the KL-divergence between the empirical distribution and a uniform-selection baseline, with full details provided in Appendix B.5.

Position bias depends on the model and the setting. Figure 3 shows a non-uniform positional selection pattern under whole-pool prompting on `Matbench Steels` that occurs to a much lesser extent under acquisition within small batches. The effect is particularly pronounced for DeepSeek-V4-Flash, whose selections concentrate near the beginning of the candidate list, with almost no selections beyond the first 100 positions. Gemma 4 31B selects from a broader portion of the list but still exhibits localized peaks. The complete KL-divergence results reported in the appendix Table 3 show that the magnitude of this non-uniformity varies substantially across models and tasks. Candidate ordering should therefore be considered a potential source of sensitivity when dealing with large contexts.

8.2 Materials Context

Prior work has suggested that LLMs can iteratively propose solutions from previously evaluated solution-value pairs and adapt their in-context priors in regression and bandit settings [Yang et al., 2024, Coda-Forno et al., 2023]. In materials science, LLMs may encode qualitative knowledge about composition-property relationships, synthesis constraints, or known alloying trends. We test whether this semantic context affects acquisition performance on `Matbench Steels` by comparing materials-aware and anonymized-feature prompts under both whole-pool and batch-tournament selection, using the same 25 initializations shown in Figure 4.

Prompt conditions. In the materials-aware condition used throughout the main experiments, candidates retain semantically meaningful feature labels and the prompt describes the materials optimization task. In the anonymized condition, the same numerical features are denoted $x_1, \ldots, x_d$, and the task is described only as maximizing an unknown scalar target. The anonymized prompt does not mention materials, alloys, elements, chemistry, or physical property names. Thus, the conditions differ in their semantic context while preserving the numerical candidate information. Prompt examples are provided in Appendix B.4.

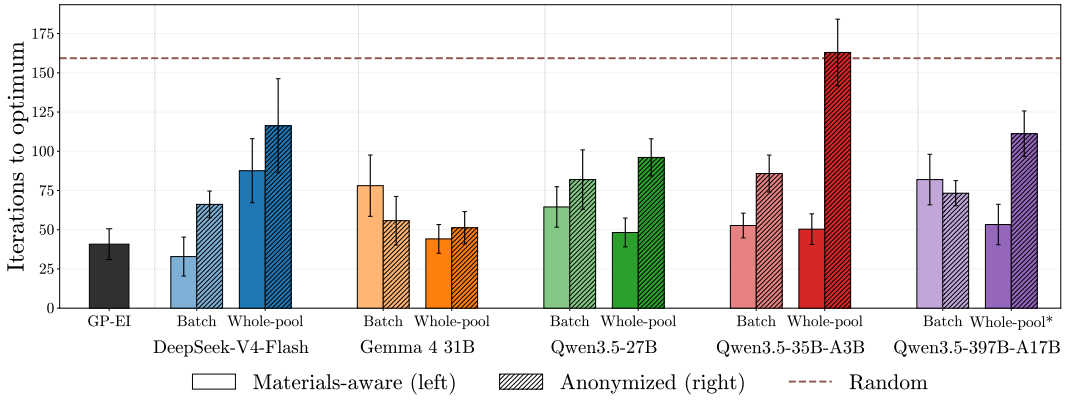


Figure 4: Optimization performance on `Matbench Steels`, with and without materials context across 25 initializations. Bars report the mean iteration at which the optimum is reached, with normal-approximation 95% confidence intervals, computed as mean $\pm\ 1.96\,\mathrm{std}/\sqrt{n_{\mathrm{seeds}}}$. We also report the mean iterations to optimum of random selection with a dotted line.

The effect of materials-aware prompting depends on the candidate-selection protocol. Figure 4 shows that LLMs retain meaningful optimization ability without explicit materials context: nine of the ten anonymized model–protocol configurations reach the optimum in fewer mean iterations than random selection. Materials-aware prompting nevertheless yields a lower mean in eight of the ten comparisons, including all five whole-pool settings. Under batch-tournament selection, anonymized prompting yields lower means for Gemma 4 31B and Qwen3.5-397B-A17B, although the confidence intervals of the two prompt conditions overlap in both cases. These results indicate that numerical features and observed outcomes alone provide a useful signal for black-box optimization, while semantic descriptions of the features and scientific objective provide additional value in many materials active-learning settings. This pattern is consistent with pretrained LLMs mobilizing materials-relevant prior knowledge when semantic context is available.

8.3 GP-Relative Acquisition Diagnostic

Examining iterations to optimum performance does not reveal what kind of acquisition policy a LLM implements. To characterize selection behavior, we investigate whether LLM selections can be compared to those of GP methods by visualizing their behavior in a two-dimensional PCA space.

Comparison with GP-like exploration-exploitation behavior. Figure 5 shows a single illustrative trajectory on `Matbench Steels`. In this run, GP-EI initially samples across the composition

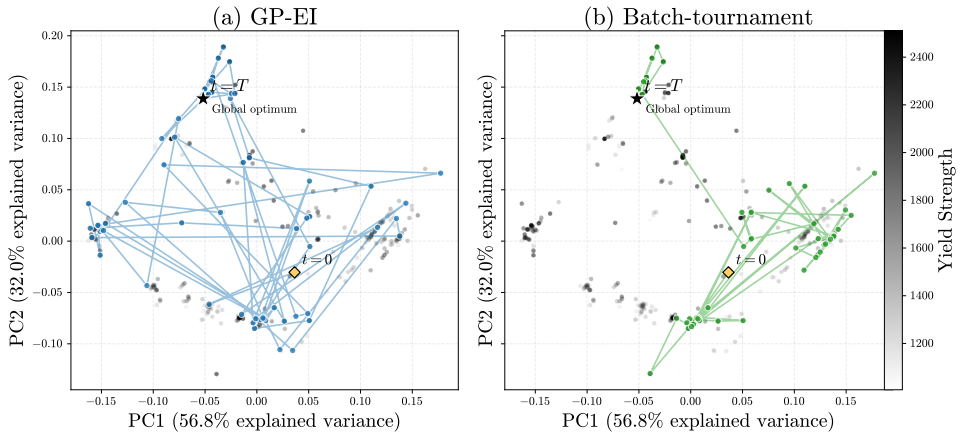


Figure 5: Explorative behavior of two selection protocols on `Matbench Steels` for seed 4 with Gemma 4 31B. Dots represent points of the pool projected onto the first two principal components, with the PCA being scaled over every pool entry. On this seed, GP-EI and batch-tournament reach the optimum after 60 and 56 iterations, respectively.

space before concentrating its selections, whereas Gemma 4 31B focuses earlier on a narrower region while still occasionally selecting distant candidates. This example suggests a more irregular transition between exploration and exploitation for the LLM policy. Even though the LLM selections are informed enough to outperform random selection, they do not resemble a regular GP-EI policy. Further analysis in Appendix D provides additional results consistent with this observation.

9 Conclusion

Across four retrospective finite-pool materials benchmarks, our results show that pretrained open-weight LLMs provide a genuine acquisition signal. LLM policies repeatedly outperform random selection, including when candidate features and objectives are anonymized. On `Matbench Steels`, adding materials-specific feature names and task descriptions improves mean performance in most evaluated settings and in every whole-pool comparison. Together, these results provide evidence that pretrained LLMs carry materials-relevant priors that can inform sequential candidate selection.

These priors do not yet produce consistently competitive acquisition policies. GP-EI remains stronger on most tasks under whole-pool selection, while LLM rankings vary across tasks and do not follow predictably from model size or architecture. Performance is also sensitive to the candidate-selection protocol, batch size, initialization, and candidate ordering, with some models exhibiting substantial position bias. The usefulness of the LLM acquisition signal therefore depends strongly on how the candidate pool and scientific context are presented.

Within our focused scope of retrospective finite-pool optimization, these findings establish standalone LLM acquisition as a credible direction for AI-guided materials discovery. The controlled comparisons developed here provide an initial empirical basis for studying how pretrained scientific priors can be translated into more reliable acquisition decisions across tasks and at larger scales.

References

Abdul Hameed Azeemi, Ihsan Ayyub Qazi, and Agha Ali Raza. Language model-driven data pruning enables efficient active learning. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 4373–4392. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-eacl.229. URL https://aclanthology.org/2026.findings-eacl.229/.

Markus Bayer, Justin Lutz, and Christian Reuter. ActiveLLM: Large language model-based active learning for textual few-shot scenarios. *Transactions of the Association for Computational Linguistics*, 14:1–22, 2026. doi: 10.1162/tacl.a.63. URL https://aclanthology.org/2026.tacl-1.1/.

Benjamin Burger, Phillip M. Maffettone, Vladimir V. Gusev, Catherine M. Aitchison, Yang Bai, Xiaoyan Wang, Xiaobo Li, Ben M. Alston, Buyi Li, Rob Clowes, Nicola Rankin, Brandon Harris, Reiner Sebastian Sprick, and Andrew I. Cooper. A mobile robotic chemist. *Nature*, 583:237–241, 2020. doi: 10.1038/s41586-020-2442-2. URL https://doi.org/10.1038/s41586-020-2442-2.

Keith T. Butler, Daniel W. Davies, Hugh Cartwright, Olexandr Isayev, and Aron Walsh. Machine learning for molecular and materials science. *Nature*, 559(7715):547–555, 2018. doi: 10.1038/s41586-018-0337-2. URL https://doi.org/10.1038/s41586-018-0337-2.

Chih-Yu Chang, Milad Azvar, Chinedum Okwudire, and Raed Al Kontar. LLINBO: Trustworthy LLM-in-the-loop bayesian optimization. *arXiv preprint arXiv:2505.14756*, 2025. doi: 10.48550/arXiv.2505.14756. URL https://arxiv.org/abs/2505.14756.

Abdoulatif Cissé, Xenophon Evangelopoulos, Vladimir V. Gusev, and Andrew I. Cooper. Language-based bayesian optimization research assistant (BORA). In James Kwok, editor, *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pages 4967–4975. International Joint Conferences on Artificial Intelligence Organization, 8 2025. doi: 10.24963/ijcai.2025/553. URL https://doi.org/10.24963/ijcai.2025/553. Main Track.

Abdoulatif Cissé, Max E. Cooper, Mengjia Zhu, Xenophon Evangelopoulos, and Andrew I. Cooper. Can we automate scientific reasoning in closed-loop experiments using large language models? *Digital Discovery*, 5(3):1132–1160, 2026. doi: 10.1039/D5DD00520E. URL https://doi.org/10.1039/D5DD00520E.

Julian Coda-Forno, Marcel Binz, Zeynep Akata, Matthew Botvinick, Jane X. Wang, and Eric Schulz. Meta-in-context learning in large language models. *arXiv preprint arXiv:2305.12907*, 2023. doi: 10.48550/arXiv.2305.12907. URL https://arxiv.org/abs/2305.12907.

Cohere. Introducing rerank 3.5: Precise AI search, December 2024. URL https://cohere.com/blog/rerank-3pt5. Accessed: 2025-09-17.

Juan J. de Pablo, Nicholas E. Jackson, Michael A. Webb, Long-Qing Chen, Joel E. Moore, Dane Morgan, Ryan Jacobs, Tresa Pollock, Darrell G. Schlom, Eric S. Toberer, James Analytis, Ismaila Dabo, Dean M. DeLongchamp, Gregory A. Fiete, Gregory M. Grason, Geoffroy Hautier, Yifei Mo, Krishna Rajan, Evan J. Reed, Efrain Rodriguez, Vladan Stevanovic, Jin Suntivich, Katsuyo Thornton, and Ji-Cheng Zhao. New frontiers for the Materials Genome Initiative. *npj Computational Materials*, 5:41, 2019. doi: 10.1038/s41524-019-0173-4. URL https://doi.org/10.1038/s41524-019-0173-4.

DeepSeek-AI et al. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.

Alexander Dunn, Qi Wang, Alex Ganose, Daniel Dopp, and Anubhav Jain. Benchmarking materials property prediction methods: The matbench test set and automatminer reference algorithm. *npj Computational Materials*, 6:138, 2020. doi: 10.1038/s41524-020-00406-3. URL https://doi.org/10.1038/s41524-020-00406-3.

Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

Daya Guo et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025. doi: 10.1038/s41586-025-09422-z. URL https://doi.org/10.1038/s41586-025-09422-z.

Anubhav Jain, Shyue Ping Ong, Geoffroy Hautier, Wei Chen, William Davidson Richards, Stephen Dacek, Shreyas Cholia, Dan Gunter, David Skinner, Gerbrand Ceder, and Kristin A. Persson. Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. *APL Materials*, 1(1):011002, 2013. doi: 10.1063/1.4812323. URL https://doi.org/10.1063/1.4812323.

Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998. doi: 10.1023/A:1008306431147. URL https://doi.org/10.1023/A:1008306431147.

Agustinus Kristiadi, Felix Strieth-Kalthoff, Marta Skreta, Pascal Poupart, Alan Aspuru-Guzik, and Geoff Pleiss. A sober look at LLMs for material discovery: Are they actually good for bayesian optimization over molecules? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 25603–25622. PMLR, 2024. URL https://proceedings.mlr.press/v235/kristiadi24a.html.

Harold J. Kushner. A new method of locating the maximum point of an arbitrary multipeak curve in the presence of noise. *Journal of Basic Engineering*, 86(1):97–106, 1964. doi: 10.1115/1.3653121. URL https://doi.org/10.1115/1.3653121.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

Turab Lookman, Prasanna V. Balachandran, Dezhen Xue, and Ruihao Yuan. Active learning in materials science with emphasis on adaptive sampling using uncertainties for targeted design. *npj Computational Materials*, 5:21, 2019. doi: 10.1038/s41524-019-0153-8. URL https://doi.org/10.1038/s41524-019-0153-8.

Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022. URL http://probml.github.io/book1.

OpenAI. OpenAI o1 System Card. Technical report, OpenAI, 2024. URL https://openai.com/index/openai-o1-system-card/.

Ritik Sachin Parkar, Jaehyung Kim, Jong Inn Park, and Dongyeop Kang. SelectLLM: Can LLMs select important instructions to annotate? *arXiv preprint arXiv:2401.16553*, 2024. doi: 10.48550/arXiv.2401.16553. URL https://arxiv.org/abs/2401.16553.

Edward O. Pyzer-Knapp, Jed W. Pitera, Peter W. J. Staar, Seiji Takeda, Teodoro Laino, Daniel P. Sanders, James Sexton, John R. Smith, and Alessandro Curioni. Accelerating materials discovery using artificial intelligence, high performance computing and robotics. *npj Computational Materials*, 8:84, 2022. doi: 10.1038/s41524-022-00765-z. URL https://doi.org/10.1038/s41524-022-00765-z.

Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

Rampi Ramprasad, Rohit Batra, Ghanshyam Pilania, Arun Mannodi-Kanakkithodi, and Chiho Kim. Machine learning in materials informatics: Recent applications and prospects. *npj Computational Materials*, 3:54, 2017. doi: 10.1038/s41524-017-0056-5. URL https://doi.org/10.1038/s41524-017-0056-5.

Bojana Ranković and Philippe Schwaller. GOLLuM: Gaussian process optimized LLMs – reframing LLM finetuning through bayesian optimization. *arXiv preprint arXiv:2504.06265*, 2025. doi: 10.48550/arXiv.2504.06265. URL https://arxiv.org/abs/2504.06265v2. Version 2.

Jonathan Schmidt, Mário R. G. Marques, Silvana Botti, and Miguel A. L. Marques. Recent advances and applications of machine learning in solid-state materials science. *npj Computational Materials*, 5:83, 2019. doi: 10.1038/s41524-019-0221-0. URL https://doi.org/10.1038/s41524-019-0221-0.

Niranjan Srinivas, Andreas Krause, Sham M. Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. *arXiv preprint arXiv:0912.3995*, 2010. doi: 10.48550/arXiv.0912.3995. URL https://arxiv.org/abs/0912.3995.

Alex Wang, Haotong Liang, Austin McDannald, Ichiro Takeuchi, and A. Gilad Kusne. Benchmarking active learning strategies for materials optimization and discovery. *Oxford Open Materials Science*, 2(1):itac006, 2022. doi: 10.1093/oxfmat/itac006. URL https://doi.org/10.1093/oxfmat/itac006.

Hongchen Wang, Rafael Espinosa Castaneda, Jay R. Werber, Yao Fehlis, Edward Kim, and Jason Hattrick-Simpers. Training-free active learning framework in materials science with large language models. *npj Computational Materials*, 2026. doi: 10.1038/s41524-026-02136-4. URL https://doi.org/10.1038/s41524-026-02136-4.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022. doi: 10.48550/arXiv.2201.11903. URL https://arxiv.org/abs/2201.11903.

Yu Xia, Subhojyoti Mukherjee, Zhouhang Xie, Junda Wu, Xintong Li, Ryan Aponte, Hanjia Lyu, Joe Barrow, Hongjie Chen, Franck Dernoncourt, Branislav Kveton, Tong Yu, Ruiyi Zhang, Jiuxiang Gu, Nesreen K. Ahmed, Yu Wang, Xiang Chen, Hanieh Deilamsalehy, Sungchul Kim, Zhengmian Hu, Yue Zhao, Nedim Lipka, Seunghyun Yoon, Ting-Hao Kenneth Huang, Zichao Wang, Puneet Mathur, Soumyabrata Pal, Koyel Mukherjee, Zhehao Zhang, Namyong Park, Thien Huu Nguyen, Jiebo Luo, Ryan A. Rossi, and Julian McAuley. From selection to generation: A survey of LLM-based active learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14552–14569. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.708. URL https://aclanthology.org/2025.acl-long.708/.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2309.03409. URL https://arxiv.org/abs/2309.03409.

Ruihao Yuan, Zhen Liu, Prasanna V. Balachandran, Deqing Xue, Yumei Zhou, Xiangdong Ding, Jun Sun, Dezhen Xue, and Turab Lookman. Accelerated discovery of large electrostrains in BaTiO3-based piezoelectrics using active learning. *Advanced Materials*, 30(7):1702884, 2018. doi: 10.1002/adma.201702884. URL https://doi.org/10.1002/adma.201702884.

A Limitations

Realistic pool sizes. Our experiments use retrospective benchmarks with candidate pools of at most 921 points, whereas real materials-discovery campaigns may involve hundreds of thousands or millions of candidates. For example, BORA's hydrogen-evolution benchmark initially contains 98,423,325 feasible experimental combinations [Cissé et al., 2025, Burger et al., 2020], while the original `Electrostrain` study considered around 605,000 compositions [Yuan et al., 2018]. Our conclusions should therefore be interpreted in light of these substantial differences in scale.

Candidate selection at scale. Scale changes the relative appeal of the candidate-selection protocols. In a very large implicit space such as BORA's, or in a continuous space, generating a promising design is natural because presenting every possible candidate is impossible. For a large explicit pool, generation-and-matching may likewise become more useful because a denser pool is more likely to contain a close match to an LLM-generated proposal, provided that matching can be performed efficiently. On the small pools considered by Wang et al. [2026] of 73 to 323 candidates, however, direct pool-based selection remains feasible and provides a useful control that does not introduce a separate learned matcher. Conversely, whole-pool prompting becomes impractical as the pool grows and may exacerbate long-context and position-bias effects. Batch-tournament limits the size of each prompt, but its number of LLM calls still grows with the pool size and may become prohibitively expensive.

Cost considerations. Inference costs also limit the use of larger proprietary models. Although such models might yield more convincing results, their API costs can quickly become prohibitive, particularly for LLM matching and batch-tournament methods. Nevertheless, these expenses should ultimately be weighed against the cost of physical experiments.

B Implementation details

B.1 Model information

Gemma 4 31B is a dense 31B-parameter model [Gemma Team, 2026]. Qwen3.5-27B is also a dense model, with 27B parameters [Qwen Team, 2026]. The remaining models use mixture-of-experts architectures: DeepSeek-V4-Flash has 284B total parameters, of which 13B are activated per token [DeepSeek-AI et al., 2026]. Qwen3.5-35B-A3B has 35B total and 3B activated parameters, while Qwen3.5-397B-A17B has 397B total and 17B activated parameters [Qwen Team, 2026].

All checkpoints were served locally with vLLM and the Qwen3.5 models use the official FP8 checkpoints. In the main experiments, temperature is set to zero, which corresponds to greedy decoding, and thinking was disabled for every model.

Qwen3.5-397B-A17B proxies Due to large inference costs and computational requirements, we could not complete all 25 seeds for each of the whole-pool Qwen3.5-397B-A17B runs. We therefore use proxies of batch-tournament with $b = 300$ indicated by *, which is close to the pool size of 312 of `Matbench Steels`, but far from the 921 of `Fe-Co-Ni`. Moreover, † denotes similar proxies of $b = 32$ instead of $b = 20$. For `Electrostrain`, we have the complete runs, so we do not need these proxies. These proxies are used in Table 2, Table 3, Figure 3b, and explain why some points are missing for Qwen3.5-397B-A17B in Figure 2.

B.2 Datasets

Electrostrain. Each Electrostrain candidate is represented by eight composition variables and seven auxiliary composition-derived descriptors. The LLM policies receive all 15 features. For GP-EI, we evaluated both the full 15-dimensional representation and the eight composition variables alone, using otherwise identical preprocessing and GP settings. In preliminary experiments, the full-feature GP-EI performed similarly to random selection with a mean iterations to optimum of 37.32 ± 22.30, instead of 18.48 ± 14.53 for the composition-only representation, which remained comparable to the LLM policies. We therefore use the composition-only GP-EI in the primary comparisons.

B.3 Policies

Gaussian process. In our implementation, the GP is refit from scratch at every loop iteration. Feature vectors are standardized coordinate-wise using the mean and standard deviation of the observed feature matrix, and the same transformation is applied to the unobserved pool. Targets are also standardized using the observed-label mean and standard deviation before computing EI. The reported EI ranking is therefore computed in normalized target units. Standard deviations are clipped below at 10^{-8} to handle zero-variance coordinates, while posterior standard deviations used inside EI are clipped below at 10^{-9} before evaluating $z_t(x)$. We use scikit-learn's `GaussianProcessRegressor` with an isotropic `Matérn` kernel and length-scale bounds $(10^{-10}, 10^{5})$; the Matérn length scale is fitted by marginal-likelihood optimization with the default L-BFGS-B optimizer and five optimizer restarts, and we do not add an explicit learned noise kernel.

Batch-tournament. At each active-learning iteration, the remaining unqueried candidates are randomly permuted and split into consecutive batches. Each full batch contains exactly b candidates, while a final remainder batch contains fewer than b when the pool size is not divisible by b. A singleton remainder is therefore treated as a one-candidate batch and advances unchanged. Every LLM call receives the same observed history. Within each batch, the candidates are displayed in a random order, and the model retains one winner by returning its displayed index. The winners are collected in batch order and are not globally reshuffled before being partitioned into the next round of batches, even though candidate order within each new LLM call is randomized again. This procedure continues until at most b candidates remain, at which point a final LLM call selects the candidate to be evaluated by the oracle. Figure 6 illustrates the procedure without the various shuffles.

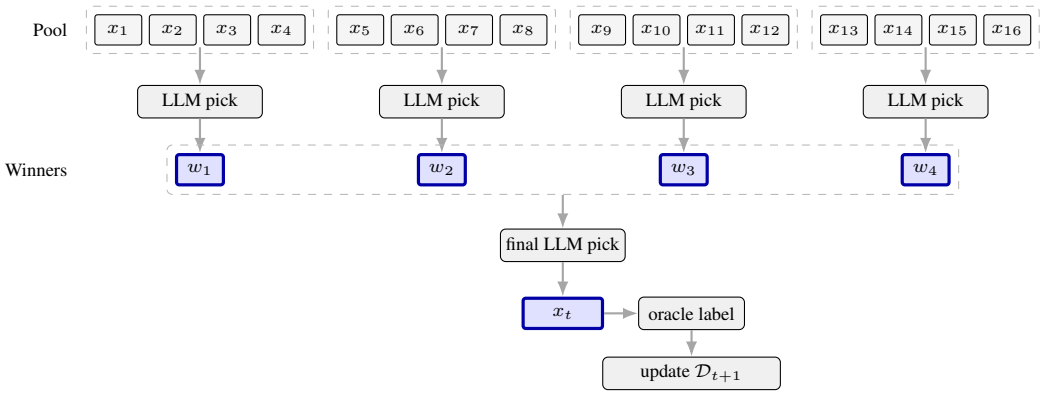


Figure 6: Batch-tournament candidate selection for an example with $N = 16$ unobserved candidates and batch size $b = 4$.

Figure 7 presents the batch-size sweeps for `Coercivity` and `Electrostrain`, whose trends are similar to those observed for the two tasks analyzed in the main text. Several `Coercivity` configurations are omitted because their inference times were prohibitively long.

B.4 Prompts.

We report below the prompt templates used for `Matbench Steels` under the materials-aware and anonymized conditions for batch-tournament selection.

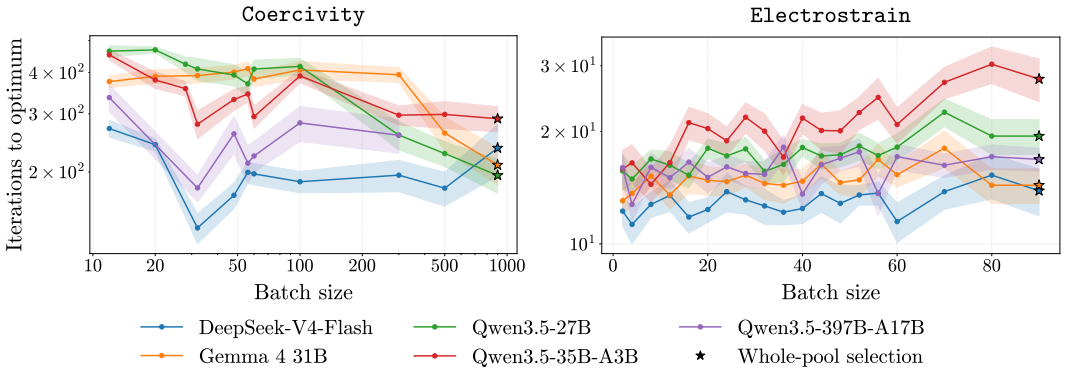


Figure 7: Batch-size sensitivity of tournament selection on `Coercivity` and `Electrostrain`. Curves report the mean number of iterations to the optimum across seeds, with uncertainty shown shaded as the mean $\pm$ std$/\sqrt{n_{\text{seeds}}}$.

B.4.1 Materials-aware.

System prompt.

```
  You are a materials scientist optimizing steel alloy compositions for maximum yield strength.

  You will be shown previously tested compositions and their yield strengths (in MPa), then a numbered
list of candidate compositions that have not been tested yet.
  Decide which candidate is most promising to test next to find high yield strength.

  **CRITICAL: Respond with ONLY the number of your chosen candidate. Nothing else.**
  Do NOT include any text, reasoning, or additional commentary.
  Your entire response must be a single number: 1, 2, 3, 4, 5, 6, 7, 8, etc.
```

User prompt.

```
  Tested compositions and their yield strengths:

  {% for comp, ys in observations -%}
  {{ loop.index }}. {{ comp }} -> {{ "%.1f"|format(ys) }} MPa
  {% endfor %}
  Candidate compositions:

  {% for comp in candidates -%}
  {{ loop.index }}. {{ comp }}
  {% endfor %}
  Which candidate should be tested next?
```

B.4.2 Anonymized

System prompt.

```
  You are an optimization algorithm searching for the global maximum of an unknown black-box function.

  You will be shown previously tested input vectors and their numeric scores, then a batch of candidate
input vectors that have not been tested yet.
  Decide which candidate is most promising to test next to maximize the score.

  **CRITICAL: Respond with ONLY the number of your chosen candidate. Nothing else.**
  Do NOT include any text, reasoning, or additional commentary.
  Your entire response must be a single number: 1, 2, 3, 4, 5, etc.
```

Table 3: Position-bias metrics for all model-task combinations. We report D_{KL} for whole-pool and batch-tournament selection with $b = 20$, except for the proxy configurations marked with an asterisk. Missing measurements are denoted by '-'.

	Electrostrain		Matbench Steels		Coercivity		Kerr Rotation	
Models	**Whole**	**Batch**	**Whole**	**Batch**	**Whole**	**Batch**	**Whole**	**Batch**
GEMMA4	0.998	2.27	1.02	2.11	1.60	2.08	2.21	2.02
DS	1.36	2.29	1.47	2.13	2.48	2.35	2.60	2.15
QW27B	0.929	2.10	0.852	2.04	2.18	2.06	2.17	2.04
QW35B	1.26	2.20	0.747	2.12	1.67	2.24	2.77	2.08
QW397B	1.04	2.20	1.36*	2.08	–	1.84†	–	2.06

User prompt.

```
  Tested input vectors and their scores:

  {% for vec, score in observations -%}
  {{ loop.index }}. {{ vec }} -> {{ "%.1f"|format(score) }}
  {% endfor %}
  Candidate input vectors:

  {% for vec in candidates -%}
  {{ loop.index }}. {{ vec }}
  {% endfor %}
  From the candidates above, choose the one most likely to produce the highest score. Answer with exactly
the number of the chosen candidate.
```

B.5 Position bias

Implementation. Because the available pool shrinks after every acquisition, raw indices are not directly comparable across iterations. For a selected candidate at one-indexed position i in a displayed pool of size K, we define its normalized position as

$$z = \frac{i-1}{K-1} \in [0, 1]. \tag{7}$$

Very small remaining pools provide fewer opportunities for position bias to appear. We therefore retain only selections made before more than 100 candidates have been removed when considering whole-pool selection, i.e., while $K \geq N - 100$ for an initial pool of size N. On batch-tournament, we are only considering requests where the LLM had to choose in a batch whose size is strictly equal to the batch size.

Let q_b be the empirical probability mass of the retained normalized positions in bin b, using a fixed set of n_{bins} equal-width bins on $[0, 1]$. We quantify departure from this uniform distribution using

$$D_{\mathrm{KL}}(q \,\|\, u) = \sum_{b=1}^{n_{\mathrm{bins}}} q_b \log \frac{q_b}{u_b} = \sum_{b=1}^{n_{\mathrm{bins}}} q_b \log(n_{\mathrm{bins}} q_b)\,. \tag{8}$$

with the convention $0 \log 0 = 0$.

We use 150 bins for the calculations reported in the table and 80 bins for the displayed plots. Because of that, we have to be careful when handling the results: we cannot directly compare batch-tournament $D_{\mathrm{KL}}(q \,\|\, u)$ values to those of whole-pool selection.

Additional results. Table 3 extends the comparison to every available model-task pair by reporting $D_{\mathrm{KL}}(q \,\|\, u)$ for every model and task. DeepSeek-V4-Flash leads almost consistently, while Gemma 4 31B remains the most consistent by always achieving reasonable KL-divergence values. Within the Qwen3.5 model family, the global behavior remains close to that of Gemma 4 31B. Regardless, trends are similar to those highlighted in the main results.

Table 4: Generation-and-matching performance on `Matbench Steels` compared with random, GP-EI, and whole-pool selection. We report iterations to the global optimum over 25 initializations (mean ± standard deviation), except for random selection which is initialized over 1000 seeds.

Model	**Random**	**GP-EI**	**Whole-pool**	**Generation-and-matching**
QW27B	159.31 ± 91.28	42.72 ± 27.24	48.24 ± 23.43	123.60 ± 63.60
GEMMA4			44.16 ± 23.36	48.80 ± 26.14

C Additional implementations

C.1 Additional policies

We have tested three other LLM policies that did not yield convincing results in our configuration. Thus, we decided to not include them in the paper.

GP+LLM. Following the general idea of hybrid LLM Bayesian optimization methods, we have tested a Bayesian optimization implementation that uses both LLM and GP-EI. At each iteration, we considered the top-p best EI samples (with p varying from 2 up to 16), and asked the LLM to choose the most promising one. It consistently gave similar results to a random choice among the points given by the GP-EI. This remark goes along those of section 8.3, as well as further experiments of Section D, which suggest that LLM selections do not follow the same exploration-exploitation pattern as GP-EI.

Swiss-tournament. We have also tried a Swiss-style tournament, where the winners of each round are paired against other winners in the next round, but it did not significantly improve performance over the batch-tournament approach, while causing longer inference times. We therefore focused on the batch-tournament and whole-pool protocols in our main experiments.

Generation-and-matching. Inspired by Wang et al. [2026], we include a generation-and-matching protocol. When presented the observed history, the LLM generates a new candidate composition, before asking a model (which might not be the same as the generating LLM) to match the generated candidate to one of the unobserved pool candidates.

Unlike Wang et al. [2026], who use the specialized Cohere Rerank-v3.5 model and its native relevance scores [Cohere, 2024], we perform matching with the same underlying model we evaluate, using normalized first-token log probabilities for `yes` and `no`.

For each generated proposal q, a reranking model compares it with every unobserved candidate x and is prompted to answer `yes` or `no` according to the similarity of their elements and fractions. We request one output token and define the matching score from its log probabilities as

$$s(q, x) = \frac{\exp(\text{logprob}(\texttt{yes}))}{\exp(\text{logprob}(\texttt{yes})) + \exp(\text{logprob}(\texttt{no}))}.$$

Missing `yes` or `no` tokens in the returned top-20 are assigned a log probability of -100, and the candidate maximizing $s(q, x)$ is selected. All pairs are scored concurrently through the vLLM OpenAI-compatible API, proposals are truncated to 500 characters and reasoning is disabled.

Table 4 reports the results on `Matbench Steels`. With Gemma 4 31B, generation-and-matching performs similarly to whole-pool selection on average, although with slightly greater variability. With Qwen3.5-27B, it requires approximately 2.6 times the mean number of iterations of whole-pool selection and also exhibits a substantially larger standard deviation. Interestingly, we do not observe the same behavior as that of Wang et al. [2026]. Under the settings evaluated here, generation-and-matching performs worse on average than both GP-EI and whole-pool selection (see Appendix A). We therefore focus on whole-pool selection in the main analysis.

C.2 Potential improvements

History management. To limit context growth, we tested an LLM-based history-management strategy that retains at most s observations by selecting which observations to keep at each iteration.

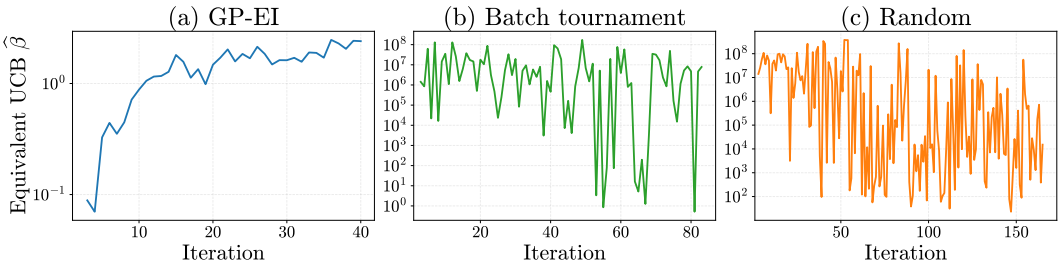


Figure 8: Comparison of evolution of fitted $\widehat{\beta}_t$ on `Matbench Steels` between Gemma 4 31B, Random and GP-EI. Results are averaged across seeds for each iteration, when at least half of the seeds have not converged yet.

This strategy performed comparably to the standard batch-tournament protocol on `Coercivity` for $s = 64$ and $s = 128$. Other selection rules, including geometric approaches, may also be worth exploring. However, we did not obtain sufficiently long runs to evaluate the method reliably beyond approximately 300 iterations, so we excluded it from the main results.

Chain-of-Thought reasoning. Chain-of-Thought prompting can improve LLM reasoning [Wei et al., 2022], but its computational cost prevented us from completing a batch-tournament evaluation. We therefore exclude it from the reported comparisons. Evaluating whether explicit reasoning improves generation-and-matching in larger candidate pools remains an avenue for future work.

D Additional diagnostics

UCB coefficient fit. To further explore the behavior of the LLM, we introduce the Upper Confidence Bound (UCB) acquisition function, which selects the candidate maximizing

$$\mathrm{UCB}_\beta(x) = \mu(x) + \sqrt{\beta}\,\sigma(x),$$

where $\mu(x)$ is the predicted performance and $\sigma(x)$ the predictive uncertainty. The parameter β controls the exploration-exploitation trade-off: small values favor exploitation, while large values favor exploration.

To quantify the exploration-exploitation trade-off of each method, we associate each decision with an equivalent UCB coefficient β: at each iteration, we fit a GP from the observations available before the decision, using the same settings as our previous GP-EI. We then estimate the value $\beta \geq 0$ for which the selected point is as close as possible to maximizing UCB:

$$\widehat{\beta}_t = \min\left(\operatorname*{argmin}_{\beta \geq 0}\left\{\max_{x \in \{1,\dots,N\}\setminus\mathcal{I}_t} \mathrm{UCB}_\beta(x_i) - \mathrm{UCB}_\beta(x_{i_t^{\mathrm{LLM}}})\right\}\right).$$

Figure 8 examines this behavior across seeds through the fitted UCB coefficient $\widehat{\beta}_t$. GP-EI exhibits a comparatively stable UCB-equivalent profile, while the coefficients fitted to Gemma 4 31B fluctuate more strongly and are closer to those obtained under random selection. Thus, although LLM selections are informed enough to outperform random selection in the main evaluation, their decisions are not consistently approximated by a fixed or smoothly evolving GP-UCB-like policy.

GP Substitution. Instead of fitting a UCB parameter, we have also tried observing the rank in terms of EI of the LLM-selected candidate after fitting a GP. Even though this method provided consistent results with those obtained previously, it was too arbitrary, which is why we decided to not include it in the main results.

E AI usage

We used AI assistants during the development and writing process, including for code prototyping, debugging, experiment-management scripts, and language polishing. All experimental results, analyses, and paper claims were checked by the authors.